Who wrote the walkthrough?

LLM attribution on gaming responses

Adam Torres | CMPSC 448 | October 11, 2026

Question

A game guide can sound convincing regardless of which model wrote it. This project asks a narrower question: can a classifier recognize the generator when the subject changes to games? The three labels are GPT-4, Llama 2 70B Chat, and Falcon 40B Instruct. A CNN and a bidirectional LSTM learn from general instruction-response pairs, and then face a gaming holdout they have never seen. Topics addressed include Zelda, poker strategies, game mechanisms, coding games, and gaming hardware. The project identifies one of three generators; it does not check the correctness of the advice.

McGovern et al. [2] noted lexical and syntactic fingerprints that could persist across domains. Their result inspired this small transfer test, but does not guarantee the classifiers will generalize. RQ1 compares response only CNN and LSTM models. RQ2 compares prompt only and mixed input models. RQ3 holds gaming material out of training. RQ4 explores the impact of common Markdown formatting.

Data and controls

UltraFeedback [1] provides prompts, names of models, and responses. I use its Evol-Instruct and UltraChat subsets, pinned to a repository revision and checked by file hashes. The original collection samples models and system instructions; labels describe generation setups, not models alone. Feedback scores and critiques never enter the classifiers. The repository includes source row IDs and prompt hashes, instead of redistributing the dataset.

A keyword search returned game candidates. Prompt review kept game strategy, development, hardware, and industry topics. Sports, figurative uses, and unrelated puzzles dressed up as games were excluded. All keyword candidates, including rejected ones, were removed from the general pool. The search has imperfect recall; it cannot prove that every remaining prompt is unrelated to games. Review membership was fixed before training.

There are 2400 responses in the training and 450 in the validation with equal class counts. The general test contains 396 responses with all three generators answering 132 prompts each. The gaming test contains 93 responses, 31 per generator, and 78 prompts. Prompt hashes keep groups disjoint before class balancing. In the general test, all prompts are fully answered by all three generators. Gaming prompts are not fully matched across generators. A prompt-only gaming result can therefore reflect the sampled prompts rather than author style. The vocabulary uses only training text. The fixed list of explicit model and vendor names is replaced by one shared token.

Models and training

Both models learn 64-dimensional embeddings using a 10,000-entry vocabulary that preserves case, punctuation, and newlines. The CNN employs 64 filters at widths 3, 5, and 7, and uses ReLU and masked maximum pooling. It follows the local-pattern approach of Kim [3], and learns embeddings from scratch. The LSTM, on the other hand, uses 64 hidden units per direction, and a single bidirectional layer. Length handling for the LSTM is tested against a packed bidirectional reference.

Each model sees a maximum of 96 prompt tokens and 160 response tokens. Combined inputs retain both token budgets plus a separator. The combined condition therefore receives more total text. Training is performed using cross-entropy, AdamW with learning rate 0.001 and weight decay 0.01, and dropout 0.3. Batches of 64 are used, and gradient clipping was set to 1.0. Validation macro-F1 selects a checkpoint over

at most 12 epochs. Patience is set to 3. 24 total runs are obtained using seeds 11, 29, and 47. All runs are performed on an Apple M4 CPU. Test results do not select settings or seeds.

Formatting is tested in two ways. The first is Markdown/whitespace stripping at test time. The second is to train the models with responses stripped of common Markdown markers and whitespace structure. Both versions use the same original response excerpt. Stripping cannot expose extra words beyond the raw token budget. This partial intervention leaves many stylistic cues and can affect code syntax. Logistic regressions on full-response length, and on length plus newline, list/heading, and code-fence counts, provide additional checks. Those baselines see the full response unlike the neural models.

Results

Accuracy for CNN response-only models was 80.6% on gaming text, compared to 74.9% for the LSTM and 33.3% chance. Their gaming macro-F1 scores were 80.7% and 75.0%. For the general test, accuracies for both models were 80.6% and 74.8% respectively. The prompt-matching conditions for the two test samples differ, therefore similar accuracies on the two test samples do not isolate the causal effect of the topic. The 95% prompt-bootstrap intervals for gaming accuracy were 73.8% to 87.1% for the CNN, and 67.0% to 82.2% for the LSTM. A stable ranking on a larger gaming corpus is not tested.

On the general test, all prompt-only models had an accuracy of 33.3%. Each identical prompt is assigned all three classes, so any deterministic guess gets one of the three responses right. On the gaming test sample, small deviations from chance can arise from the unmatched sample.

This control helps to separate response information from topic cues. Confusion counts show that among their off-diagonal errors, both response-only models most often misclassify Llama responses as GPT-4. This pattern by itself does not reveal the phrases that caused the mistakes.

Adding the prompt raises CNN gaming accuracy slightly to 82.1%, but lowers the LSTM to 63.8%. Extra context does not consistently help in this setup. Concatenation also increases sequence length, so this comparison cannot separate the added context from the effect of processing a longer sequence.

Model	Input	General: acc. / F1	Gaming: acc. / F1
CNN	Prompt	33.3 / 31.6	34.1 / 32.2
CNN	Response	80.6 / 80.6	80.6 / 80.7
CNN	Both	81.5 / 81.5	82.1 / 82.2
LSTM	Prompt	33.3 / 33.0	36.9 / 36.6
LSTM	Response	74.8 / 74.9	74.9 / 75.0
LSTM	Both	63.9 / 63.8	63.8 / 63.5

Table 1. Accuracy and macro-F1, in percent, averaged over seeds 11, 29, and 47. General n = 396; gaming n = 93. Macro-F1 gives each generator equal weight.

Formatting and failure modes

A length-only logistic regression achieves a gaming accuracy of 52.7%. Adding newline, list/heading and code-fence counts increases it to 66.7%. This shows that some responses can be easily classified with readily available information. Removing formatting at test time decreases the accuracy of the CNN and LSTM to 55.2% and 71.7%, respectively, from 80.6% and 74.9%, respectively. This edit is more detrimental to the CNN. Retraining on stripped responses tests whether useful attribution cues remain after the edit becomes part of the training distribution. Neither of the methods identifies a permanent model signature.

Retraining on stripped text brings CNN gaming accuracy back to 76.3%; the LSTM reaches 71.7%. Useful cues survive the edit. The CNN's large test-time drop therefore overstates how much formatting is necessary when the classifier can learn from edited text.

Response condition	CNN gaming accuracy	LSTM gaming accuracy
Original	80.6%	74.9%
Stripped only at test	55.2%	71.7%
Stripped and retrained	76.3%	71.7%

Table 2. The ablation preserves the original response excerpt. It removes common Markdown markers and whitespace structure, not all possible style cues.

Lessons and limits

The most instructive implementation checks were for the experiment itself. Splitting responses would allow the same prompt on both sides of the evaluation. Removing newlines before truncation would give the stripped condition additional content. Tests guard against both errors, as well as padding changes and vocabulary leakage. Fourteen automated tests pass, including a small learning check.

The gaming set is small, mixes several kinds of game-related writing, and differs from the matched general set in how prompts are sampled. Uncertainty intervals resample whole prompts and retain their responses and all three seed predictions. These intervals describe the held out sample conditional on one training split. They do not describe uncertainty with new training splits. System prompts and generation settings are potential alternative explanations for recognizable styles. A better follow-up would collect the same game prompts from every generator under the same instructions, then test unseen game genres and model versions.

References

[1] Cui et al. UltraFeedback: Boosting Language Models with Scaled AI Feedback. arXiv:2310.01377.

Dataset: openbmb/UltraFeedback, revision 40b4365. Hugging Face dataset card.

[2] McGovern, Stureborg, Suhara, and Alikaniotis. Your Large Language Models Are Leaving Fingerprints. 2024. arXiv:2405.14057.

[3] Kim. Convolutional Neural Networks for Sentence Classification. 2014. arXiv:1408.5882.

Assistance: Codex assisted with design, code, experiments, analysis and drafting. I reviewed all AI-assisted code and can explain the tradeoffs chosen. Factual claims were checked against saved outputs.